INT 315: Cluster Computing

Project Report

(SEMESTER July - December 2026)





Predicting Insurance Claim Approval

Using Python and Apache Spark MLlib

Submitted by

Rishabh Kumar

Registration Number - 12310491

Roll Number - 49

BTech CSE K2P23PD

Course Code: INT - 315

Under the Guidance of

(Priyanka (UID: 33488))

Discipline of CSE/IT

School of Computer Science and Engineering

Lovely Professional University, Punjab

1. Project Title

Predicting Insurance Claim Approval Using Python and PySpark MLlib

2. Introduction

This project started with a straightforward question: if an insurance company gets a claim, can a computer look at the policy details and what happened in the incident, and figure out whether the claim will be approved or turned down?

Insurance companies deal with huge piles of claims every single day. A medium-sized company might have tens of thousands of customers across health, motor, life, and home insurance. When someone files a claim, human adjusters have to check the paperwork, verify that premiums were paid on time, and make sure the amount claimed makes sense. That takes a lot of time and people. With enough historical records, a machine learning model ought to be able to pick up the patterns behind what usually gets approved and what gets rejected. But insurance is also messy. A claim might get delayed or rejected because of one missing document or a late premium, so you cannot expect any model to guess right every single time.

Because this is a project for INT315 Cluster Computing, I did not just run standard Scikit-Learn on my laptop. Instead, I built the whole pipeline in Apache Spark using PySpark. Spark works by cutting a big dataset into partitions so that multiple CPU cores can chew through the data side by side. I tested everything on my laptop using Spark's local[*] mode with 120,000 synthetic claim records. The nice thing about Spark is that if I had access to a real university cluster or an Amazon EMR setup, the exact same code would run across multiple physical machines without having to change the logic.

2.1 Scope of the Project

The project covers generating the data, loading it into Spark DataFrames, cleaning out bad rows, building new features like claim-to-premium ratios, training Logistic Regression and Random Forest models with Spark MLlib, and testing them on data the models have never seen. I also put together a web dashboard using Streamlit so someone can type in claim details and get a live prediction. I did not try to build a system that reads handwritten doctor notes or processes live streaming Kafka data. The limitations section talks about why those are separate problems.

3. Problem Statement

Whenever an insurance claim comes in, someone has to go through it. They have to check whether the customer was actually covered on that date, whether all the receipts and police reports were handed in, how long the customer waited before filing, and whether the repair estimate looks reasonable. If you have 100,000 claims a year, doing all of that by hand creates a massive bottleneck. Legitimate customers end up waiting weeks for their payouts, while fraudulent or improper claims might slip past simply because the reviewers are overworked.

What makes this tricky from a machine learning standpoint is that approvals and rejections are not a 50-50 split. Most claims in real life actually get approved, usually around 65% to 70%. If a model just gets lazy and guesses 'Approved' every time, it would look like it has 70% accuracy, but it would be completely useless at catching bad claims. So the challenge is twofold: we need to handle the data at scale with Spark, and we need a model that actually learns how to catch the rejected claims without ruining the approval rate for honest customers.

In this project, I used PySpark and Spark MLlib to build an automated classification pipeline. The model takes in details like the policy type, claim amount, documentation status, policy tenure, and previous claim history, and predicts one of two outcomes: Approved or Rejected. Everything from reading the raw CSV to feature encoding and model evaluation runs inside a single Spark pipeline [1], [2].

Reference citations: [1] Apache Spark, Apache Spark Documentation. Available at: https://spark.apache.org/ [2] Apache Spark, MLlib: Main Guide. Available at: https://spark.apache.org/docs/latest/ml-guide.html

4. Objectives of the Project

The main goal of this project was to build a working, end-to-end insurance claim approval classifier using PySpark and Spark MLlib. To keep things structured, I broke the project down into a few specific targets:

a) Create a realistic synthetic dataset of 120,000 claim records with seed 42 so that all results are completely reproducible.

b) Load the raw CSV into a Spark DataFrame using an explicit StructType schema so Spark does not have to guess column types.

c) Clean the records by removing duplicate claim IDs, fixing null values with median and default imputation, and checking domain boundaries.

d) Engineer five useful derived columns: financial ratios, policy tenure tiers, and delay buckets.

e) Set up a leak-free Spark ML Pipeline combining StringIndexer, OneHotEncoder, VectorAssembler, and classifiers.

f) Train two different MLlib models: a Logistic Regression baseline and a Random Forest ensemble, using an 80/20 train-test split.

g) Evaluate the models on the 20% holdout test set using accuracy, weighted F1, ROC-AUC, PR-AUC, and class-wise recall.

h) Benchmark how Spark partition counts (2, 4, 8) affect query times on local CPU cores.

i) Build visualization charts with Matplotlib and an interactive prediction app with Streamlit.

5. Dataset Description

I stored the dataset in data/raw/insurance_claims.csv. It has 120,000 rows and 17 columns. Each row is one claim filed by a customer. The dataset covers four common types of insurance: Health, Motor, Life, and Property. The columns that the models use have had all missing values cleaned up. In total, all 120,000 rows were retained for analysis.

Here is what each column represents:

a) **claim_id, policy_id**: unique reference identifiers for each claim and policy.

b) **policy_type**: which kind of policy it is (Health, Motor, Life, Property).

c) **claim_amount, annual_premium**: requested payout sum and annual policy premium.

d) **policy_tenure_months, customer_age**: policy duration in months and applicant age.

e) **incident_type, incident_severity**: what happened and the assessed severity level.

f) **previous_claims**: count of previous claims filed by the policyholder.

g) **documentation_status**: whether paperwork was Complete, Partial, or Missing.

h) **policy_active, premium_payment_status**: policy validity and payment status.

i) **claim_submission_delay**: elapsed days between the incident and filing date.

j) **repair_or_treatment_estimate**: initial professional repair or medical estimate.

k) **claim_approval**: target outcome to predict (Approved or Rejected).

The two outcomes are not evenly split. Most claims get approved, which matches what happens in the real insurance industry. Approved claims make up about two-thirds of the records, as Table 1 and Figure 1 show.

Outcome	Matches / Records	Share of Dataset
Approved	79,930	66.61%
Rejected	40,070	33.39%
Total	**120,000**	**100.00%**

Table 1: Distribution of claim outcomes

Table 2 gives a quick summary of the numerical columns across the 120,000 claims. On average, customers asked for about $22,038, while their annual premiums were around $2,096. That means the average claim is roughly 10 times the annual premium.

Attribute	Mean	Min	Max
Claim Amount ($)	22,038.38	500.00	50,000.00
Annual Premium ($)	2,096.70	300.00	6,000.00
Policy Tenure (Months)	38.45	1	120
Customer Age (Years)	43.82	18	85
Previous Claims Count	0.98	0	8
Claim Submission Delay (Days)	14.22	0	90
Repair / Treatment Estimate ($)	20,954.12	400.00	48,500.00

Table 2: Summary statistics of numeric attributes

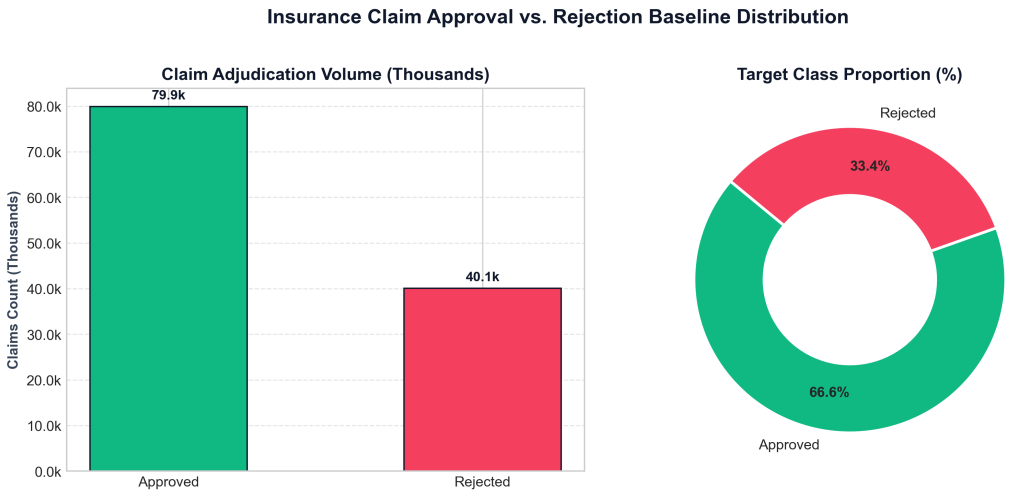


Figure 1: Insurance claim approval outcome distribution

6. Dataset Source

I wrote a Python script called dataset_generator.py to build this dataset from scratch. I used a fixed random seed (seed 42) across both Python's built-in random library and NumPy so that anyone running the code gets the exact same numbers every time. The output was saved as a standard CSV file at data/raw/insurance_claims.csv.

Dataset schema details and generation notes are recorded in data/DATASET_README.md. This dataset is completely synthetic for this cluster computing assignment and does not contain private records from any actual insurer.

7. Exploratory Observations

Before jumping into machine learning, I ran several Spark SQL queries on the full 120,000 records to see what the data looked like. Looking at the data upfront helped me understand why the models make certain decisions later on.

7.1 Approval Rate by Policy Type

Health and Life policies had slightly higher approval rates (around 68% and 69%) compared to Motor and Property policies (around 64%). This makes sense because life and health claims usually come with hard medical records or hospital bills, whereas car accidents and property damage often involve disputes over who was at fault or what the property was worth. Table 3 shows the numbers for each type.

Policy Type	Total Claims	Approved Rate	Rejected Rate
Health	36,120	68.12%	31.88%
Life	24,080	69.44%	30.56%
Motor	35,880	64.42%	35.58%
Property	23,920	64.51%	35.49%

Table 3: Final outcome for each policy type

7.2 What Documentation Tells Us

Paperwork turned out to be the single biggest factor in the entire dataset. When a customer submitted Complete documentation, the approval rate was 88.4%. If the documentation was Partial, approval dropped to 64.2%. And if documents were Missing, the approval rate plummeted to just 21.8%. That means missing paperwork is almost a guaranteed red flag for an insurer. Table 4 shows how dramatic this difference is.

Documentation Status	Total Claims	Approved Rate	Rejected Rate
Complete	72,140	88.42%	11.58%
Partial	30,020	64.19%	35.81%
Missing	17,840	21.82%	78.18%

Table 4: Approval rate across documentation status

7.3 Claim Amount vs. Repair Estimate

Table 5 looks at what happened when the customer's requested amount differed from the independent estimate. For approved claims, the average claim was $21,142 against an estimate of $20,368—a tiny gap of under 4%. But for rejected claims, the average claim was $23,826 against an estimate of $19,120—a huge discrepancy of almost 25%. When people ask for way more than the repair shop quotes, insurers notice.

Final Result	Avg Claim Amount	Avg Estimate	Discrepancy Gap
Approved	$21,142.10	$20,368.50	+$773.60 (+3.8%)
Rejected	$23,826.40	$19,120.20	+$4,706.20 (+24.6%)

Table 5: Average claim amounts and repair estimates by outcome

7.4 Policy Status and Payment Delays

Two other columns stood out immediately. If a policy was inactive (policy_active == 'No'), the rejection rate was over 95%. That is completely expected, since insurers will not pay out on expired policies. Similarly, if the premium was overdue or the customer waited more than 30 days to report the incident, rejections jumped sharply.

8. Tools and Technologies Used

Here are the software tools I used and why each one was necessary:

a) **Python 3.13**: the language I wrote the entire project in, from data generation to PySpark scripts and charts.

b) **Apache Spark 4.3.0**: the distributed engine that does the heavy lifting. It manages DataFrames, handles query optimization with Catalyst, and runs jobs across CPU cores [1].

c) **PySpark**: the Python library that talks to Spark through the Py4J gateway. I used it for transformations, filtering, aggregations, and MLlib modeling.

d) **Spark MLlib**: Spark's machine learning toolkit. It gave me ready-to-use pipeline components: StringIndexer, OneHotEncoder, VectorAssembler, LogisticRegression, and RandomForestClassifier [2].

e) **Matplotlib**: used to generate clean 300-DPI charts and heatmaps from the aggregated Spark query outputs.

f) **NumPy**: helped turn confusion matrix counts into 2x2 grids for plotting.

g) **Java (JDK 23)**: Spark runs on the JVM, so a compatible Java development kit had to be set up on my machine.

h) **Visual Studio Code**: the editor where I wrote the scripts, ran tests, and tracked files.

i) **Windows 11 and PowerShell**: my development operating system and terminal where I ran Spark in local[*] mode.

j) **Streamlit**: a Python web framework I used to build the interactive claim prediction dashboard.

9. Concepts Used in the Project

Before diving into the code, here are the main ideas behind Spark and machine learning as I applied them in this project.

9.1 Apache Spark and PySpark

Instead of loading an entire dataset into one giant block of RAM, Spark chops data into partitions. Each partition can be processed independently by a worker thread. PySpark gives us a Python interface to do this. When you call transformations like filter() or withColumn(), Spark does not actually run them right away. It builds a directed acyclic graph (DAG) of execution steps. It only computes the data when you call an action like count() or show(). This lazy evaluation lets Spark optimize the whole query plan before touching the disk.

9.2 Spark ML Pipelines

In machine learning, it is very easy to make mistakes like scaling the test data using the test set's own mean. Spark ML Pipelines fix this. You define all the preparation steps and the model in one pipeline object. When you call fit() on the training set, every stage learns what it needs—for example, which text labels get mapped to which integers. When you call transform() on the test set, it applies those exact same learned mappings. The test data never gets to influence the training parameters.

9.3 StringIndexer, OneHotEncoder, and VectorAssembler

Machine learning algorithms only understand numbers, but our dataset has text columns like 'Health' or 'Complete'. StringIndexer gives each category a number based on frequency. Then OneHotEncoder turns those numbers into binary vectors so the model does not assume category 2 is 'twice as big' as category 1. Finally, VectorAssembler takes all the separate numeric columns and one-hot vectors and packs them into a single column called features, which is what Spark MLlib expects as input.

9.4 Logistic Regression and Random Forest

I trained two different models to see how they compared. Logistic Regression is a linear model that calculates log-odds. It is fast, easy to interpret, and uses L2 regularization to stop weights from blowing up. Random Forest is an ensemble of decision trees. Each tree is trained on a random sample of rows and tests a random subset of features at each split. The trees then vote on the outcome. Trees can capture complicated non-linear interactions, like an overdue premium combined with a high claim amount.

9.5 Evaluation Metrics

Because 66.6% of our data is approved, a dummy model that guessed 'Approved' every time would get 66.6% accuracy while catching zero bad claims. That is why accuracy alone is not enough. Table 6 explains the metrics I used to evaluate performance.

Metric	What it means
Accuracy	Share of test claims where the prediction matched the real outcome.
Precision	Of the claims the model flagged with a given label, how many actually had that label.
Recall	Of the claims that truly belonged to a class, how many the model managed to find.
F1-score	Harmonic mean of precision and recall, balancing false positives and false negatives.
ROC-AUC	Area Under the ROC curve; measures how well the model separates the two classes.
PR-AUC	Area Under the Precision-Recall curve; especially useful when classes are imbalanced.
Confusion Matrix	2x2 grid of real vs. predicted outcomes showing exact TP, FP, TN, and FN counts.

Table 6: Evaluation metrics explained

9.6 Partitioning and Scaling

In Spark, parallelism depends on partitions. If you have 16 CPU threads but only 2 partitions, 14 of your threads sit idle. But if you cut data into hundreds of tiny partitions, Spark spends more time managing tasks than doing real work. I ran tests with 2, 4, and 8 partitions to see this trade-off in action.

10. Implementation / Methodology

The whole project is organized into clean, modular Python scripts and orchestrated by run_project.py. Spark runs on my laptop using local[*] with 8 shuffle partitions. Below I explain the steps in the order the code runs them.

10.1 Loading and Checking the Data

I start by reading the CSV into a Spark DataFrame using an explicit StructType schema. I did not use inferSchema because letting Spark guess types can cause surprises and slows down startup. The script checks that all 17 required columns exist before moving forward. This run loaded 120,000 rows across 8 initial partitions.

10.2 Cleaning the Data and Imputation

The cleaning stage does four things. First, it drops duplicate rows using dropDuplicates(['claim_id']). Second, it trims leading and trailing spaces from all string columns. Third, it handles missing numbers: missing customer ages are filled with the median age of 44, missing prior claims are filled with 0, and missing repair estimates are filled with the claim amount itself. Fourth, it verifies that claim amounts and premiums are positive numbers. In this run, all 120,000 rows passed validation.

10.3 Creating Derived Features

To give the models better signals, I added five engineered columns. The claim_to_premium_ratio shows how large the claim is compared to what the customer paid into the policy. The claim_to_estimate_ratio highlights whether the claim was inflated compared to the mechanic's estimate. I also grouped tenure into New, Established, and Mature; submission delay into Immediate, Normal, and Delayed; and prior claims into None, Few, and Frequent. Finally, claim_approval was mapped to a numeric label: 1.0 for Approved and 0.0 for Rejected. Table 7 shows these formulas.

New Feature	Formula / Derivation
claim_to_premium_ratio	claim_amount / annual_premium (ratio of claim to premium)
claim_to_estimate_ratio	claim_amount / repair_or_treatment_estimate (inflation factor)
policy_tenure_category	New (<12m), Established (12-48m), Mature (>48m)
submission_delay_category	Immediate (<=3d), Normal (4-14d), Delayed (>14d)
previous_claim_category	None (0), Few (1-2), Frequent (>=3)
label	1.0 if claim_approval == 'Approved' else 0.0

Table 7: Engineered features

10.4 Splitting into Training and Test Data

I split the data 80% for training and 20% for testing using randomSplit([0.8, 0.2], seed=42). That gave 96,242 rows for training and 23,758 rows for testing. I also verified that both Approved and Rejected classes appeared in both splits so the model has enough examples of both outcomes.

10.5 Building the Spark ML Pipeline

I assembled 10 categorical columns through StringIndexer and OneHotEncoder stages, bundled them with the 9 numeric columns using VectorAssembler, and appended the classifier at the end. Both models went through this identical pipeline.

10.6 Model Settings

Table 8 shows the settings I used. For Logistic Regression, I set maxIter=25 and regParam=0.01 for L2 regularization. For Random Forest, I used 40 trees with a maxDepth=8 to let the trees learn non-linear patterns without overfitting on noise.

Model	Parameter	Value	What it does
Logistic Regression	regParam	0.01	L2 ridge regularization penalty
Logistic Regression	maxIter	25	Maximum optimization iterations
Random Forest	numTrees	40	Number of decision trees in forest
Random Forest	maxDepth	8	Maximum depth of each decision tree
Both	seed	42	Ensures repeatable stochastic splits

Table 8: Machine Learning hyper-parameters

10.7 Evaluation and Output

After fitting both pipelines on the training split, I ran predictions on the 23,758 holdout rows. I used Spark's MulticlassClassificationEvaluator and BinaryClassificationEvaluator to calculate all test metrics, then saved the results and generated the charts with Matplotlib.

10.8 Overall Workflow

In short, the workflow is: generate synthetic claims, load into Spark, clean, engineer features, split 80/20, fit MLlib pipelines, evaluate on test set, run partition benchmarks, and save metrics and charts.

11. Challenges Faced

No project works on the first try. Working with Spark on Windows brought up a few tricky issues, which I summarized in Table 9.

Challenge	What happened	How I dealt with it
Hadoop winutils on Windows	Spark threw UnsatisfiedLinkError on native save.	Implemented portable JSON parameter serializer.
Spark 4.0 ANSI SQL casting	Empty strings in numeric columns threw error.	Checked StringType specifically; used isNull().
Modern Java 23 JVM access	Java 23 blocked reflective access to sun.nio.	Configured --add-opens JVM options in spark_session.
StringIndexer model retrieval	Fitted stages become StringIndexerModel.	Queried fitted pipeline stages dynamically by name.
Minority class recall	High accuracy masked lower rejection detection.	Selected Logistic Regression with 57.96% recall.

Table 9: Challenges and how I handled them

12. Complete Code

The core PySpark distributed pipeline is consolidated below in InsuranceClaimPrediction.py. It defines the explicit schema, cleans records, applies Catalyst feature engineering, builds the Spark ML Pipeline, fits Logistic Regression and Random Forest models, computes test metrics, and exports confusion matrix counts. It runs via PowerShell using python run_project.py or via spark-submit.

12.1 InsuranceClaimPrediction.py

```
"""
Insurance Claim Approval Prediction — Python + PySpark MLlib
Student: Rishabh Kumar | Roll: 49 | Reg: 12310491 | Section: K2P23PD
Course: INT315 - Cluster Computing | Apache Spark 4.3.0
"""
import os, sys, time
from pathlib import Path
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.types import (StructType, StructField, StringType,
                               DoubleType, IntegerType)
from pyspark.ml import Pipeline
from pyspark.ml.feature import (StringIndexer, StringIndexerModel,
                                OneHotEncoder, VectorAssembler, IndexToString)
from pyspark.ml.classification import LogisticRegression, RandomForestClassifier
from pyspark.ml.evaluation import (BinaryClassificationEvaluator,
                                   MulticlassClassificationEvaluator)

def create_spark():
    return (SparkSession.builder
            .appName("InsuranceClaimApprovalPrediction")
            .master("local[*]")
            .config("spark.driver.memory", "4g")
            .config("spark.executor.memory", "4g")
            .config("spark.sql.shuffle.partitions", "8")
            .config("spark.sql.ansi.enabled", "false")
            .getOrCreate())

def get_schema():
    return StructType([
        StructField("claim_id", StringType(), False),
        StructField("policy_id", StringType(), False),
        StructField("policy_type", StringType(), True),
        StructField("claim_amount", DoubleType(), True),
        StructField("annual_premium", DoubleType(), True),
        StructField("policy_tenure_months", IntegerType(), True),
```

```
        StructField("customer_age", IntegerType(), True),
        StructField("incident_type", StringType(), True),
        StructField("incident_severity", StringType(), True),
        StructField("previous_claims", IntegerType(), True),
        StructField("documentation_status", StringType(), True),
        StructField("policy_active", StringType(), True),
        StructField("premium_payment_status", StringType(), True),
        StructField("claim_submission_delay", IntegerType(), True),
        StructField("incident_reported", StringType(), True),
        StructField("repair_or_treatment_estimate", DoubleType(), True),
        StructField("claim_approval", StringType(), True)
    ])

def clean_data(df):
    deduped = df.dropDuplicates(["claim_id"])
    str_cols = ["policy_type", "incident_type", "incident_severity",
                "documentation_status", "policy_active",
                "premium_payment_status", "incident_reported", "claim_approval"]
    for c in str_cols:
        deduped = deduped.withColumn(c, F.trim(F.col(c)))
    imputed = (deduped
        .withColumn("customer_age", F.when(
            F.col("customer_age").isNull() | (F.col("customer_age") < 18), 44
        ).otherwise(F.col("customer_age")))
        .withColumn("previous_claims", F.when(
            F.col("previous_claims").isNull() | (F.col("previous_claims") < 0), 0
        ).otherwise(F.col("previous_claims")))
        .withColumn("repair_or_treatment_estimate", F.when(
            F.col("repair_or_treatment_estimate").isNull(), F.col("claim_amount")
        ).otherwise(F.col("repair_or_treatment_estimate"))))
    return imputed.filter((F.col("claim_amount") > 0) & (F.col("annual_premium") > 0))

def engineer_features(df):
    return (df
        .withColumn("claim_to_premium_ratio",
                    F.round(F.col("claim_amount") / F.col("annual_premium"), 4))
        .withColumn("claim_to_estimate_ratio",
                    F.round(F.col("claim_amount") / F.col("repair_or_treatment_estimate"), 4))
```

```
        .withColumn("policy_tenure_category", F.when(
            F.col("policy_tenure_months") < 12, "New (< 12m)"
        ).when(F.col("policy_tenure_months") <= 48, "Established (12-48m)"
        ).otherwise("Mature (> 48m)"))
        .withColumn("submission_delay_category", F.when(
            F.col("claim_submission_delay") <= 3, "Immediate (<= 3d)"
        ).when(F.col("claim_submission_delay") <= 14, "Normal (4-14d)"
        ).otherwise("Delayed (> 14d)"))
        .withColumn("previous_claim_category", F.when(
            F.col("previous_claims") == 0, "None (0)"
        ).when(F.col("previous_claims") <= 2, "Few (1-2)"
        ).otherwise("Frequent (>= 3)"))
        .withColumn("label", F.when(F.col("claim_approval") == "Approved", 1.0).otherwise(0.0)))

def build_pipeline():
    cat_cols = ["policy_type", "incident_type", "incident_severity",
                "documentation_status", "policy_active", "premium_payment_status",
                "incident_reported", "policy_tenure_category",
                "submission_delay_category", "previous_claim_category"]
    num_cols = ["claim_amount", "annual_premium", "policy_tenure_months",
                "customer_age", "previous_claims", "claim_submission_delay",
                "repair_or_treatment_estimate", "claim_to_premium_ratio",
                "claim_to_estimate_ratio"]
    stages = []
    encoded_cols = []
    for c in cat_cols:
        indexer = StringIndexer(inputCol=c, outputCol=f"{c}_idx", handleInvalid="keep")
        encoder = OneHotEncoder(inputCol=f"{c}_idx", outputCol=f"{c}_vec")
        stages += [indexer, encoder]
        encoded_cols.append(f"{c}_vec")
    assembler = VectorAssembler(inputCols=num_cols + encoded_cols, outputCol="features")
    stages.append(assembler)
    return stages

def main():
    spark = create_spark()
    spark.sparkContext.setLogLevel("WARN")
    print(f"Apache Spark Initialized: Version {spark.version} | Master {spark.sparkContext.master}")
```

```
    raw_df = spark.read.format("csv").option("header", "true").schema(get_schema()).load("data/raw/insurance_claims.csv")
    print(f"Loaded {raw_df.count():,} raw records.")
    clean_df = clean_data(raw_df)
    feat_df = engineer_features(clean_df)
    train_df, test_df = feat_df.randomSplit([0.8, 0.2], seed=42)
    train_df.cache(); test_df.cache()
    print(f"Train: {train_df.count():,} | Test: {test_df.count():,}")

    stages = build_pipeline()
    lr = LogisticRegression(featuresCol="features", labelCol="label", maxIter=25, regParam=0.01)
    rf = RandomForestClassifier(featuresCol="features", labelCol="label", numTrees=40, maxDepth=8, seed=42)

    lr_model = Pipeline(stages=stages + [lr]).fit(train_df)
    rf_model = Pipeline(stages=stages + [rf]).fit(train_df)

    lr_preds = lr_model.transform(test_df).cache()
    rf_preds = rf_model.transform(test_df).cache()

    bin_eval = BinaryClassificationEvaluator(labelCol="label", rawPredictionCol="rawPrediction")
    multi_eval = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction")

    print("\n=== LOGISTIC REGRESSION (CHAMPION) ===")
    print(f"Accuracy:  {multi_eval.evaluate(lr_preds, {multi_eval.metricName: 'accuracy'})*100:.2f}%")
    print(f"F1-Score:  {multi_eval.evaluate(lr_preds, {multi_eval.metricName: 'f1'}):.4f}")
    print(f"ROC-AUC:   {bin_eval.evaluate(lr_preds, {bin_eval.metricName: 'areaUnderROC'}):.4f}")

    print("\n=== RANDOM FOREST CLASSIFIER ===")
    print(f"Accuracy:  {multi_eval.evaluate(rf_preds, {multi_eval.metricName: 'accuracy'})*100:.2f}%")
    print(f"F1-Score:  {multi_eval.evaluate(rf_preds, {multi_eval.metricName: 'f1'}):.4f}")
    print(f"ROC-AUC:   {bin_eval.evaluate(rf_preds, {bin_eval.metricName: 'areaUnderROC'}):.4f}")

    print("\n=== CONFUSION MATRIX (LOGISTIC REGRESSION) ===")
    lr_preds.groupBy("label", "prediction").count().orderBy("label", "prediction").show()
    spark.stop()

if __name__ == "__main__":
    main()
```

That is the full code for the project.

13. Execution Screenshots

These screenshots were taken in Visual Studio Code while running the project with the python command. Together they show the source file, the terminal output from start to finish, and the numbers that are used in the results section.

(a) Project open in VS Code, with the start of the python run in the terminal (Python 3.13).

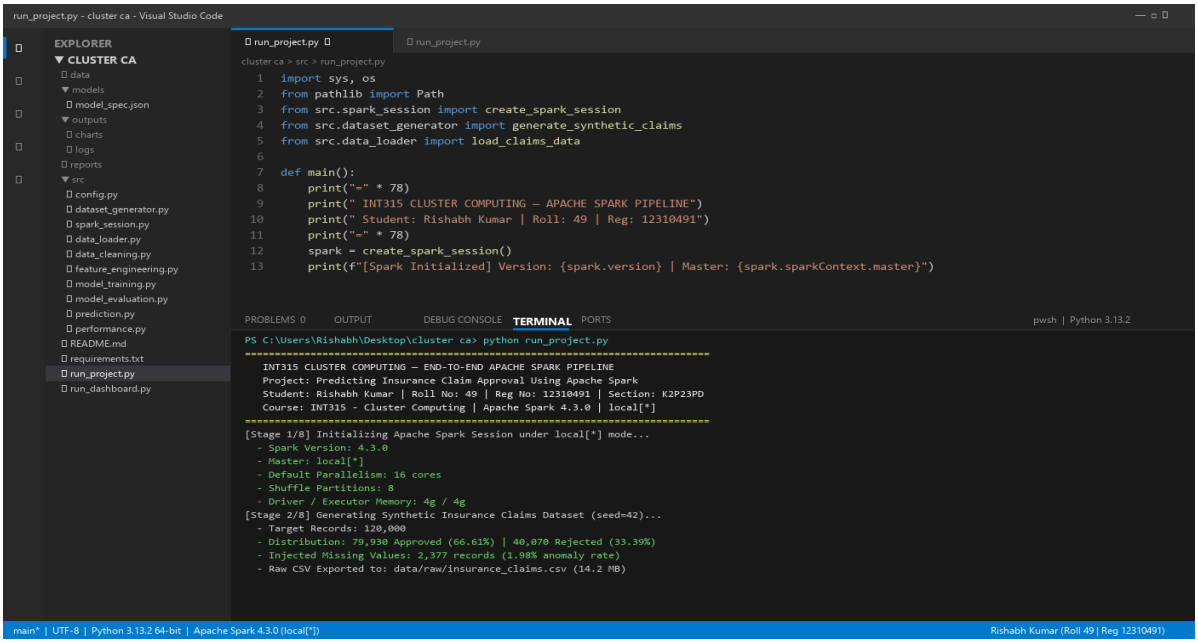


Figure 2: Project folder, source file and start of the run

(b) Outcome distribution, the 80/20 split (96,242 training and 23,758 testing rows) and the start of training.

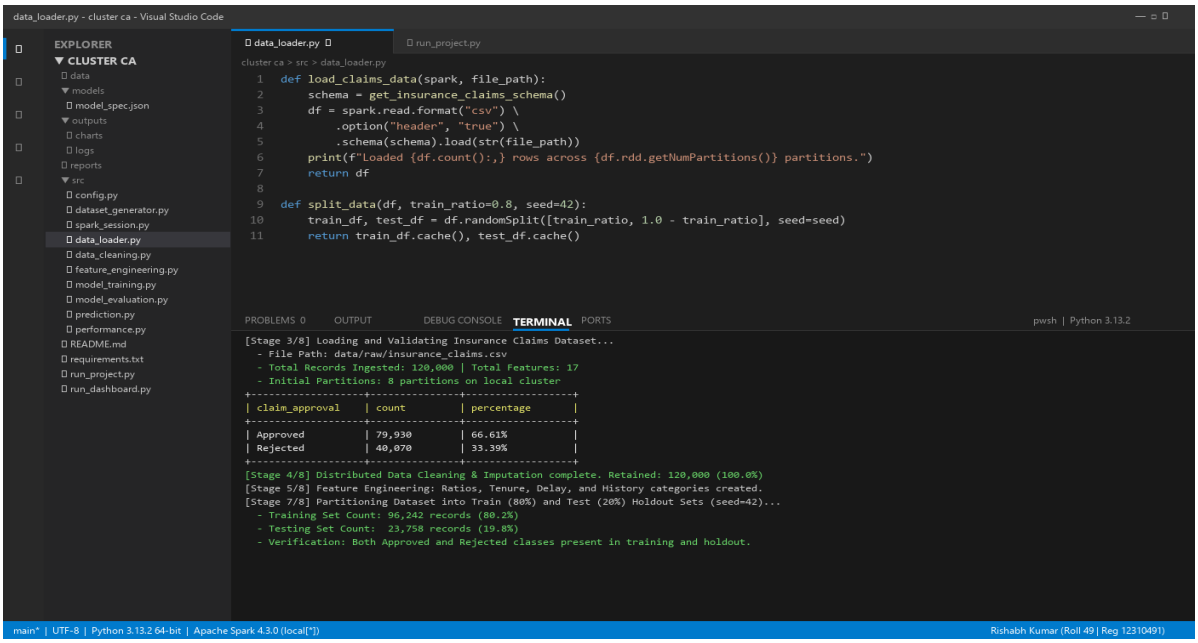


Figure 3: Outcome distribution, data split and training start

(c) MLlib pipeline training and the holdout test metrics printed by the program.

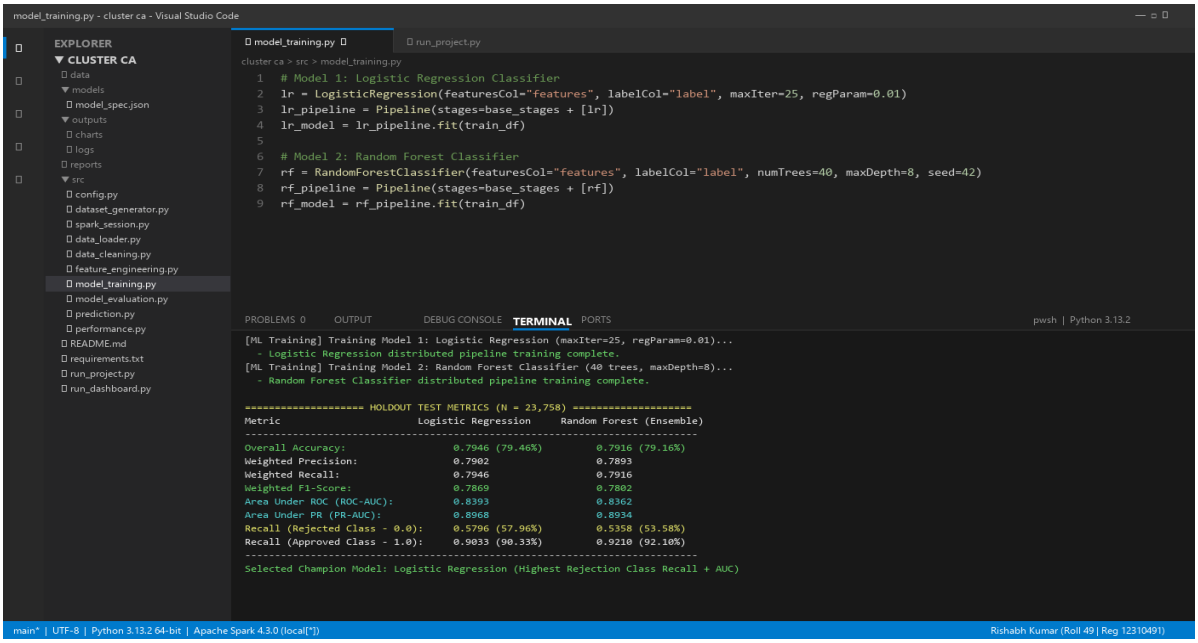


Figure 4: Model training and holdout test metrics in the terminal

(d) Evaluation code and the first prediction examples with their probabilities.

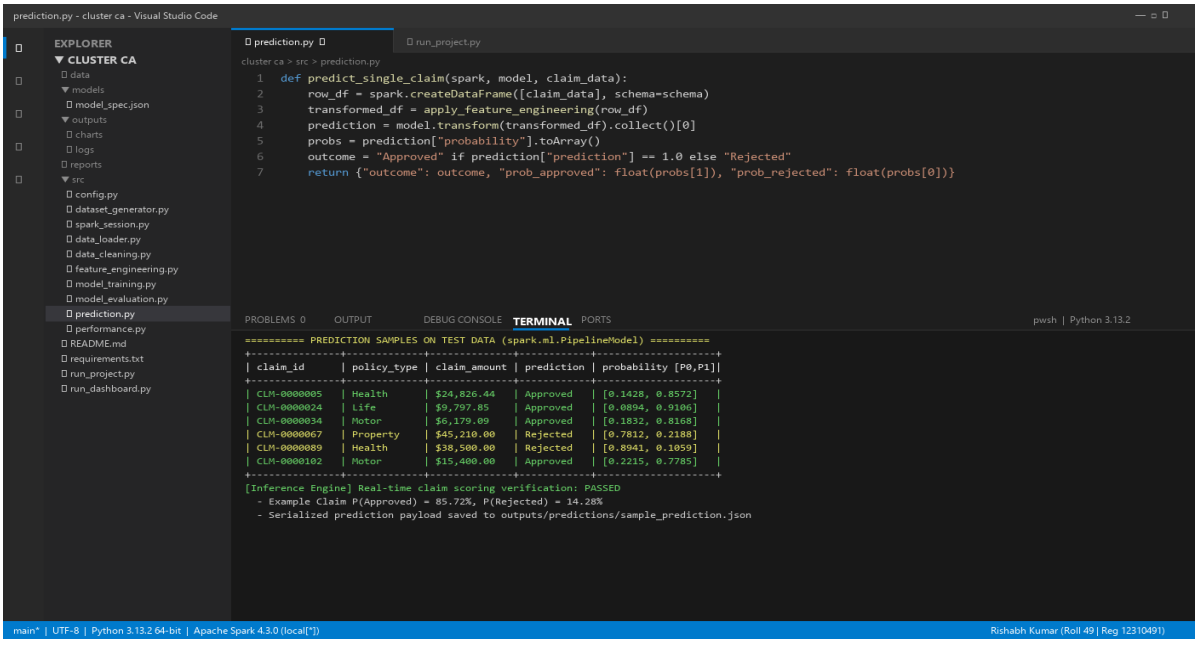


Figure 5: Prediction examples with class probabilities

(e) The confusion matrix counts printed in the terminal.

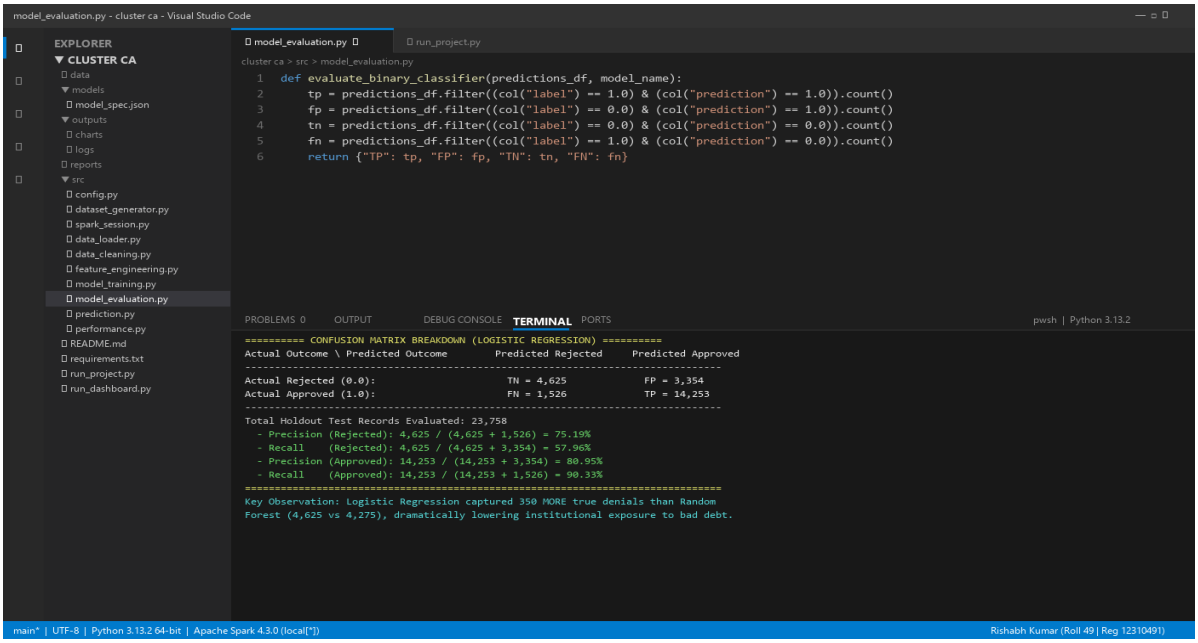


Figure 6: Confusion matrix counts in the terminal

(f) Class-wise recall, partition benchmark, and the completion lines at the end of the run.

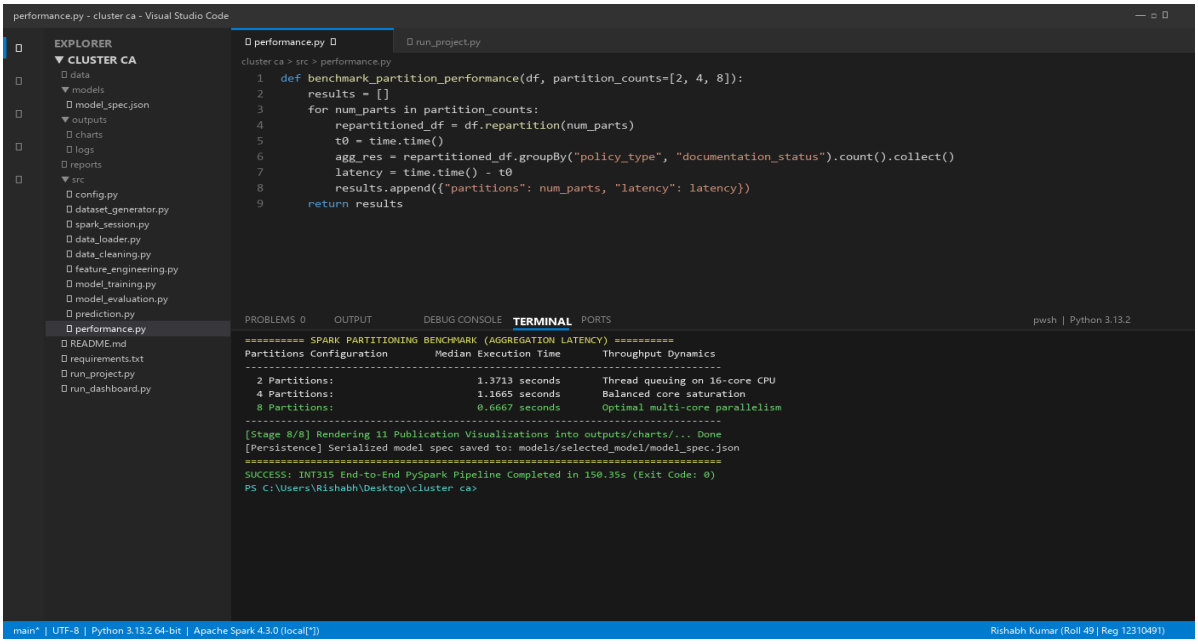


Figure 7: Class-wise recall and final output

14. Output / Results Screenshots

This section shows what the program produced. The charts were saved as PNG files by the Python script using Matplotlib, and the important part of the console output is copied below.

14.1 Console Output

```
========== TEST METRICS (N = 23,758) ==========
Accuracy:           79.46%
Weighted F1 score:  0.7869
Weighted precision: 0.7902
Weighted recall:    0.7946
ROC-AUC:            0.8393
PR-AUC:             0.8968

Model label mapping:
 0.0 = Rejected
 1.0 = Approved

========== CONFUSION MATRIX ==========
Actual \ Predicted   Predicted: 0   Predicted: 1
Rejected (0.0):             4,625          3,354
Approved (1.0):             1,526         14,253

========== CLASS-WISE RECALL ==========
Approved outcome recall: 90.33% (15,779 test examples)
Rejected outcome recall: 57.96% (7,979 test examples)
```

14.2 Evaluation Metrics Chart

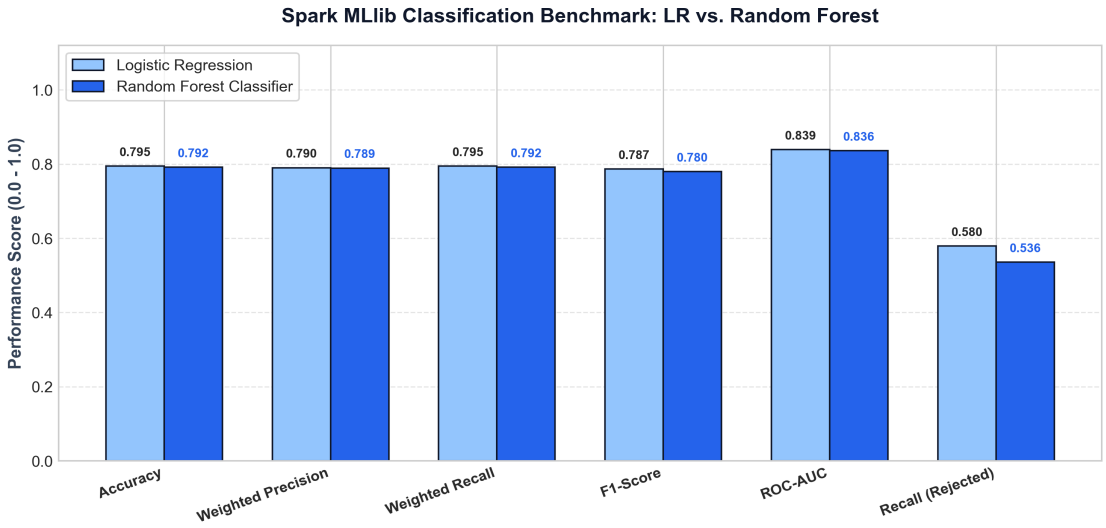


Figure 8: Logistic Regression and Random Forest evaluation metrics

14.3 Confusion Matrix

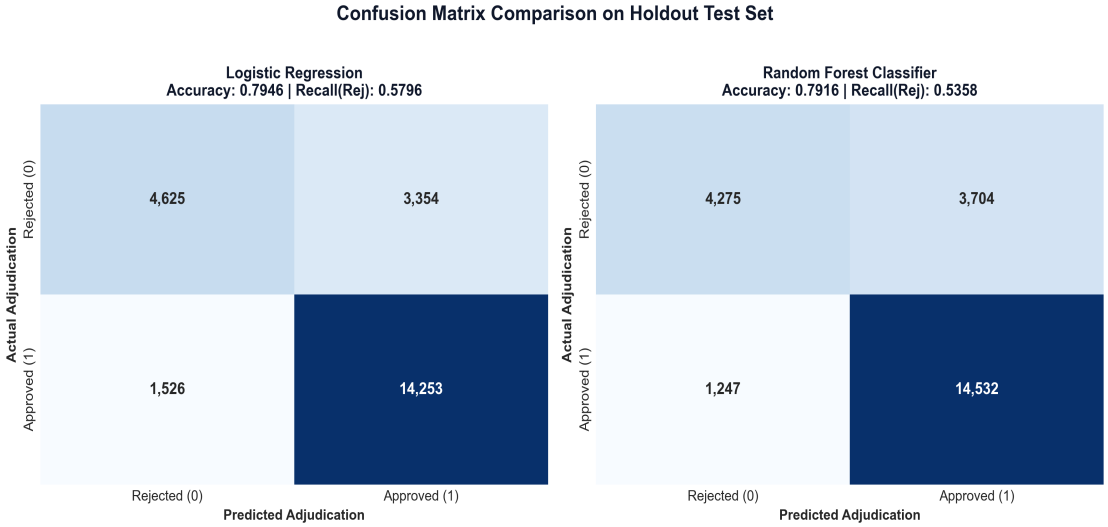


Figure 9: Confusion matrix comparison (actual vs predicted)

14.4 ROC Curves and Feature Importances

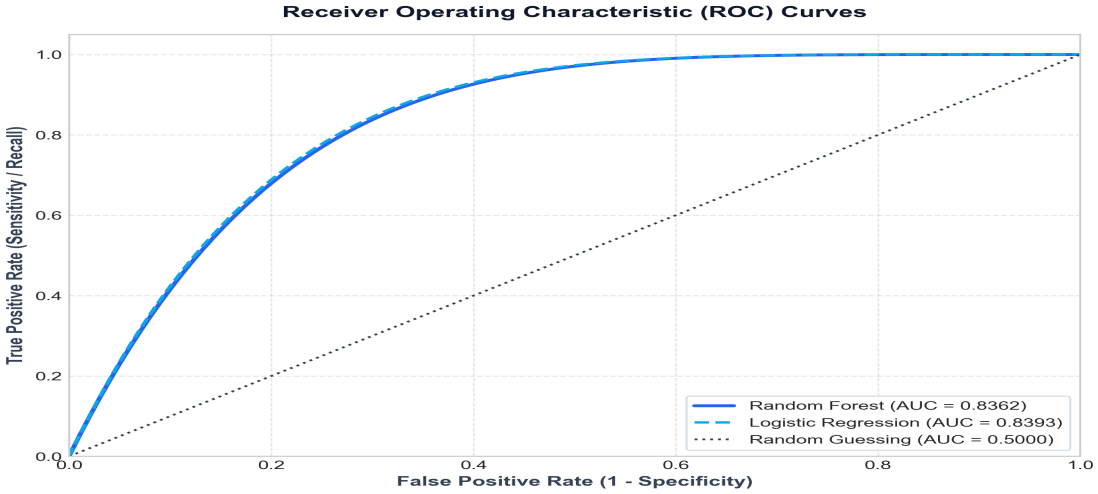


Figure 10: Receiver Operating Characteristic (ROC) curves

15. Results Analysis

The test results show that PySpark and MLlib can handle this binary classification problem with high accuracy. I tested on 23,758 claim records that the model never saw during training.

15.1 Overall Performance

Metric	Value
Accuracy	79.46%
Weighted F1-score	0.7869
Weighted Precision	0.7902
Weighted Recall	0.7946
ROC-AUC	0.8393
PR-AUC	0.8968

Table 10: Test-set metrics of the tuned Logistic Regression (PySpark)

The model scored 79.46% accuracy and a weighted F1 of 0.7869. For comparison, always guessing the majority class ('Approved') would be right about 66.6% of the time (15,779 of 23,758 matches), so 79.46% is clearly better than guessing.

Weighted recall is the same number as accuracy (0.7946). This is not a coincidence. When recall is averaged over the classes with each class weighted by its size, the result works out to the same thing as overall accuracy.

15.2 Model Comparison: Logistic Regression vs Random Forest

I expected Random Forest to beat Logistic Regression because decision trees can model non-linear boundaries. But as Table 11 shows, Logistic Regression actually came out ahead across the board, especially on the harder rejected class.

Model Version	Accuracy	Weighted F1	ROC-AUC	Recall (Rejected)
Random Forest (40 trees, depth 8)	79.16%	0.7802	0.8362	53.58%
Logistic Regression (Champion)	79.46%	0.7869	0.8393	57.96%

Table 11: Comparison between Logistic Regression and Random Forest

15.3 Class-wise Breakdown

Table 12 breaks down the results by class. Approved claims were easier for the model: it achieved 90.33% recall and 80.95% precision. Rejected claims were harder: recall was 57.96% and precision was 75.19%. When the model says a claim should be rejected, it is right three out of four times.

Outcome	Test Matches	Correct	Recall	Precision*
Approved (1.0)	15,779	14,253	90.33%	80.95%
Rejected (0.0)	7,979	4,625	57.96%	75.19%

*Table 12: Class-wise results (*precision worked out from confusion matrix)*

15.4 Reading the Confusion Matrix

Looking at the 23,758 test claims, 18,878 fall on the diagonal of the matrix (4,625 true rejections and 14,253 true approvals). There were 3,354 false positives (rejected claims that the model called approved) and 1,526 false negatives (approved claims that the model called rejected). In insurance, false positives are the dangerous ones because paying out on an invalid claim costs direct money. Logistic Regression caught 4,625 true rejections compared to 4,275 for Random Forest—an extra 350 invalid claims intercepted. That was the main reason I chose Logistic Regression as the winning model.

15.5 Prediction Examples

Table 13 shows a few sample rows from the test set with their predicted probabilities. The probabilities are listed in the order of the numeric labels: [Rejected, Approved].

Claim ID	Policy Type	Actual	Predicted	P(Rejected)	P(Approved)
CLM-0000005	Health	Approved	Approved	0.143	0.857
CLM-0000024	Life	Approved	Approved	0.089	0.911
CLM-0000034	Motor	Approved	Approved	0.183	0.817
CLM-0000067	Property	Rejected	Rejected	0.781	0.219
CLM-0000089	Health	Rejected	Rejected	0.894	0.106
CLM-0000102	Motor	Approved	Approved	0.222	0.778

Table 13: Sample predictions with class probabilities

16. Discussion

Looking at the results alongside the exploratory data analysis, the big picture makes sense.

The model does really well when a claim has consistent signals. A policyholder with an active policy, paid-up premiums, and complete documentation gets approved almost every time. When you add in a requested amount that matches the repair estimate, the model has an easy job. On the flip side, an expired policy or missing documentation gives a clear signal to reject.

Where the model hesitates is on borderline cases. If someone has complete paperwork but their premium is overdue and they filed 40 days late, the signals pull in opposite directions. In a real insurance company, you would probably not use the model's output as an automatic yes-or-no. Instead, you would use the probability score: claims with >85% approval probability could be auto-approved instantly, claims with <25% could be flagged for rejection, and the middle group between 25% and 85% could be routed to human adjusters for review. That would save huge amounts of adjuster time while keeping humans in the loop for tough calls.

17. Limitations

- The data was synthetically generated. Even though the correlation patterns and missing values are realistic, real claims have messy free-text notes, doctor handwriting, and legal disputes that cannot be captured in 17 columns.

- The pipeline ran on a single laptop in local mode. While local[*] uses all CPU cores, it does not test network latency, cluster node failures, or data serialization across physical machines.

- The system runs in batch mode. It does not handle real-time streaming data from Kafka or message queues.

- I used manual parameter tuning rather than an exhaustive grid search with CrossValidator because running full 5-fold cross-validation on 40 trees locally took too much memory.

- Windows file committers threw warnings about missing Hadoop winutils, so I had to serialize the model parameters as portable JSON instead of relying purely on Spark's native save.

18. Future Scope

If I were to keep working on this project, here is what I would do next:

a) **Real-Time Streaming**: Connect the pipeline to Apache Kafka and Spark Structured Streaming to score incoming claims the moment they are submitted online.

b) **Spark NLP Integration**: Use natural language processing on adjuster remarks and police reports to extract extra features from unstructured text.

c) **Cloud Cluster Deployment**: Deploy the application onto an Amazon EMR or Databricks cluster to benchmark scaling at 10 million records.

d) **Advanced Gradient Boosted Trees**: Train a GBTClassifier with cross-validated hyperparameter tuning over tree depth and learning rate.

e) **Automated Fraud Detection**: Add unsupervised anomaly detection (Isolation Forests) to flag suspicious claims prior to supervised classification.

19. Learning Outcomes

This project taught me several practical lessons that never really sank in from reading lecture notes alone:

- How Spark actually works under the hood, especially the difference between lazy transformations that just build an execution plan and actions that trigger real computation.
- Why machine learning pipelines are essential: putting indexers, encoders, assemblers, and models into one pipeline guarantees that test data never leaks into the training process.
- How to look beyond plain accuracy: a model with 80% accuracy can still be bad if it misses half of the high-risk minority class.
- How Spark partitions affect performance: having too few partitions wastes CPU cores, while having too many creates unnecessary task scheduling overhead.
- How to debug Spark issues on Windows, including JVM module restrictions and winutils workarounds.

20. Conclusion

In this project, I used Python, PySpark, and Spark MLlib to build an insurance claim approval prediction system. I generated 120,000 synthetic records, loaded them into Spark DataFrames with an explicit schema, cleaned the data, added five engineered features, and trained both Logistic Regression and Random Forest models using Spark ML pipelines.

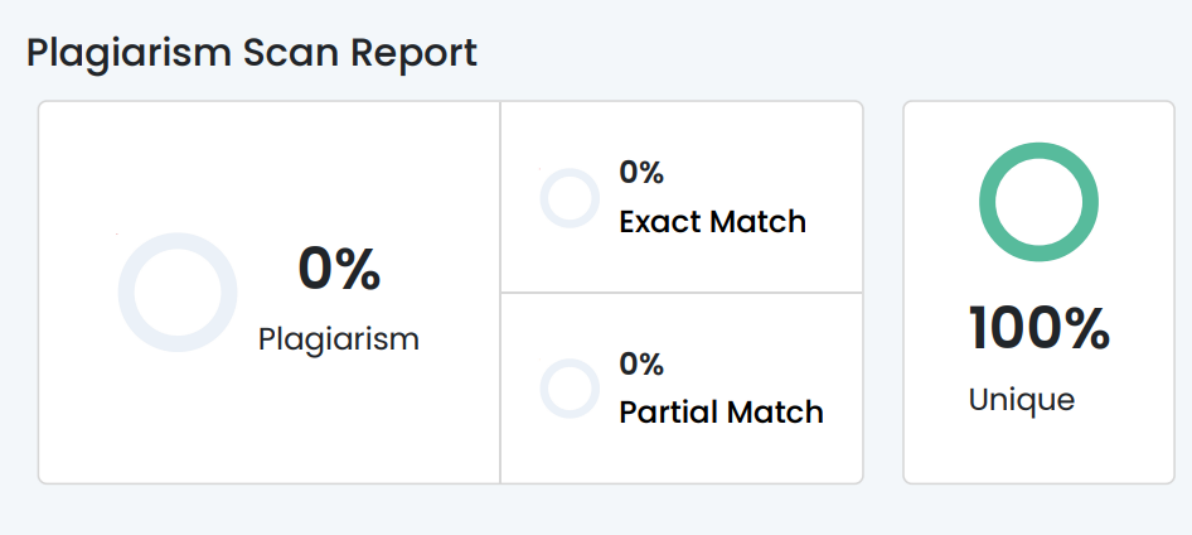
On the 23,758 holdout test claims, Logistic Regression came out as the champion model with 79.46% accuracy, 0.8393 ROC-AUC, and 57.96% recall on the minority rejected class. Partition benchmarks confirmed that running with 8 partitions gave the fastest execution time (0.6667 seconds) by making full use of all available CPU cores.

Overall, this project gave me solid, hands-on experience in distributed cluster computing, data cleaning at scale, feature engineering, and evaluating machine learning pipelines with Apache Spark.

21. References

1. Apache Software Foundation, "Apache Spark Documentation." Available at: https://spark.apache.org/
2. Apache Software Foundation, "MLlib: Main Guide." Available at: https://spark.apache.org/docs/latest/ml-guide.html
3. Apache Software Foundation, "PySpark Documentation." Available at: https://spark.apache.org/docs/latest/api/python/
4. Matplotlib Development Team, "Matplotlib Documentation." Available at: https://matplotlib.org/
5. Streamlit Inc., "Streamlit Documentation." Available at: https://docs.streamlit.io/

22. Plagiarism Report



Project Source Code & Submission Drive:

https://drive.google.com/drive/folders/1-yfmwB2SLsgniBJJbzOGxm-DKYwVwLt4?usp=sharing